AI Smart Supply Chain Optimizer Report

Submitted By

Muhammad Saad Sohail — 24K-0549 (Leader)

Khadija Abbasi — 24K-0770

Muhammad Abdullah — 24K-0977

Course: Artificial Intelligence | Instructor: Riaz Ahmed

1. SUMMARY

The AI Smart Supply Chain Optimizer is a full-stack intelligent logistics platform that automates delivery planning through a multi-layered AI pipeline. Users submit delivery requests in plain English; the system parses them with a Large Language Model, builds a real-world route graph via OpenRouteService, finds optimal paths with A* search, assigns vehicles through a Genetic Algorithm, validates the plan against capacity and deadline constraints using CSP, and returns a natural-language summary.

The project demonstrates solid integration of four major AI techniques: **heuristic search, optimization, constraint satisfaction, and language model inference.** The codebase is clean, modular, and production-oriented with FastAPI, MongoDB, and a CORS-enabled REST API with OpenAI integration.

2. SYSTEM ARCHITECTURE

The system follows a nine-step end-to-end pipeline:

Step	Component	File
1	User Input (natural language)	FastAPI – main.py /llm-inputs
2	LLM Parsing & Constraint Extraction	llm_service.py (Groq LLaMA-3.3-70b)
3	Persistence	MongoDB (pymongo)
4	Graph Construction	map_api_service.py (OpenRouteService API)
5	Route Optimisation	astar_service.py (A* with Manhattan heuristic)
6	Vehicle Assignment & Scheduling	ga_service.py (Genetic Algorithm)
7	Constraint Validation	csp_service.py (Coverage, Capacity, Deadlines)
8	Plan Summarisation	llm_service.py – summarize_plan()
9	Frontend Display	index.html (vanilla JS + CSS)

3. AI TECHNIQUES IN DEPTH

3.1 Large Language Model — Input Parsing & Summarisation

The LLM layer uses the Groq-hosted LLaMA-3.3-70b-versatile model via an OpenAI-compatible client. Two separate calls are made:

- parse_input() — A strict extraction prompt instructs the model to return a JSON object with fields: source, destinations, deadline, budget, objective, packages, vehicle_type, avoid, and constraints. Temperature is set to 0 for determinism.

- summarize_plan() — A summarisation prompt converts the structured optimisation output into bullet-point plain English for the end user.

The _safe_json_loads() helper strips Markdown code fences, finds the JSON object boundaries, and raises a ValueError on malformed output — a pragmatic guard against LLM hallucination of non-JSON text.

3.2 Graph Modelling — OpenRouteService Integration

map_api_service.py constructs a weighted directed graph of all location pairs. For each origin-destination pair it calls the ORS /v2/directions/driving-car endpoint, extracting distance (km) and duration (min). Cost is derived as distance × cost_per_km. Geocoding appends 'Karachi, Pakistan' to every query and applies a geographic bounding box (24.7N–25.3N, 66.9E–67.4E) to restrict results to the city. A request throttle and exponential-backoff retry (3 attempts, doubling delay) prevent rate-limit errors.

3.3 A* Search — Route Cost Optimisation

astar_service.py implements textbook A* over the adjacency-dictionary graph. Key design decisions:

- Heuristic: Manhattan distance on (lat, lon) coordinates, falling back to 0 when coordinates are unavailable — admissible in a grid-like city layout.
- Priority queue: Python's heapq min-heap on f = g + h.
- Metrics: On path reconstruction, _calculate_metrics() accumulates cost, distance, and time across all edges.
- Edge cost: Uses the 'distance' field first; falls back to 'cost' if absent.

A* runs for every origin-destination pair across all delivery nodes, populating the distance_lookup table consumed by the GA.

3.4 Genetic Algorithm — Vehicle & Route Assignment

ga_service.py is the most substantial AI component (~250 lines). A chromosome encodes three parallel lists: routes (stops per vehicle), vehicles (type), and route types (highway / expressway / city).

GA Component	Implementation
Population size	50 chromosomes (configurable)
Generations	200 (configurable)
Mutation rate	0.1 per operator
Selection	Tournament selection (size 3)
Crossover	Order-preserving single-point on flattened route list

GA Component	Implementation
Mutation ops	Location swap, relocation, shuffle, vehicle/route type flip
Fitness	Weighted cost + time + penalty (capacity, coverage, fragile/medical constraints)

Vehicle types (small truck , medium truck , truck , heavy truck , refrigerated truck) and route factors (highway x1.0, expressway x0.8, city x1.2) are defined as constants. The fitness function applies domain-specific penalties: +30 for fragile goods in heavy vehicles, +50 for medical supplies not in a refrigerated truck. main.py runs the GA 5 times and selects the best valid plan.

3.5 Constraint Satisfaction Problem — Plan Validation

csp_service.py enforces four constraint classes after the GA produces a plan:

- Coverage: Every delivery location appears exactly once across all vehicle routes.
- Capacity: Per-vehicle package totals must not exceed the vehicle's rated capacity.
- Deadlines: Cumulative travel time from depot to each stop is compared to the parsed deadline. A 15-minute service time per stop is added. Supports HH:MM, '2h', and plain-minute formats.
- Route Consistency: No vehicle visits the same location consecutively.

validate_with_details() returns a structured dictionary with per-check pass/fail and detailed violation lists, suitable for API consumption.

4. CODE QUALITY & ARCHITECTURE ASSESSMENT

Dimension	Assessment
Modularity	Each AI technique isolated in its own service file; main.py only orchestrates.
Type Annotations	(Dict, List, Optional, Tuple) throughout all service files.
Error Handling	ValueError, HTTPException, retry logic, JSON-fence stripping — comprehensive.
Separation of Concerns	API layer, business logic, and external integrations cleanly separated.
Naming Conventions	Consistent snake_case; function names clearly express intent.
Hardcoded Constants	Vehicle capacities duplicated in GA and CSP; should share a single source.
Test Coverage	No test files found in the zip; unit tests absent.

Dimension	Assessment
Configuration	Environment variables used for API keys and tunable params.

5. REST API ENDPOINTS

Method	Endpoint	Purpose
GET	/	Health check
GET	/ui	Serves index.html frontend
POST	/test-llm	Parse natural language input via LLM (no persistence)
POST	/llm-inputs	Parse input and store in MongoDB
GET	/llm-inputs	List all stored parsed inputs
DELETE	/llm-inputs	Clear all stored inputs
GET	/llm-process	Run full pipeline on stored inputs, return LLM summary
POST	/route	Run A* between two points with provided coordinates

6. STRENGTHS

- Real-world data: OpenRouteService integration produces genuine distance and travel-time data rather than synthetic approximations.
- LLM robustness: Strict extraction prompt, temperature-0 setting, and JSON fence stripping make the NLP layer production-worthy.
- GA multi-run: Running the GA 5 times and picking the best valid result reduces sensitivity to random initialisation.
- Karachi-aware geocoding: Bounding box and focus-point bias prevent the geocoder returning locations outside the target city.
- Request throttling: ORS rate-limiter with exponential backoff is a professional touch rarely seen in academic projects.
- Deadline parsing: Multi-format deadline parser handles real-world input diversity (HH:MM, AM/PM, hours, plain minutes).

7. Future Enhancements

- **More Vehicle Types:** Add bikes, mini-vans, drones, and cargo trucks with different capacities and fuel costs.
- **More Route Types:** Include rural roads, toll roads, flooded routes, and night-only roads with different speed/cost factors.

- **Multi-Warehouse Support:** Allow deliveries from multiple warehouses instead of one central warehouse.
- **Real Traffic Data:** Use live traffic and rush-hour data through ORS for smarter routing.
- **Priority Deliveries:** Add high/medium/low priority deliveries in GA optimization.
- **Driver Shift Constraints:** Prevent routes that exceed driver working-hour limits.
- **Return Trip Optimization:** Include vehicle return trips in total route cost calculations.
- **Inventory Checking:** Verify warehouse stock before planning deliveries.
- **Live Re-routing:** Add a `/replan` endpoint for route updates during delays or road closures.
- **Dashboard & Analytics:** Track delivery performance, costs, traffic patterns, and GA statistics using MongoDB.

8. TECHNOLOGY STACK SUMMARY

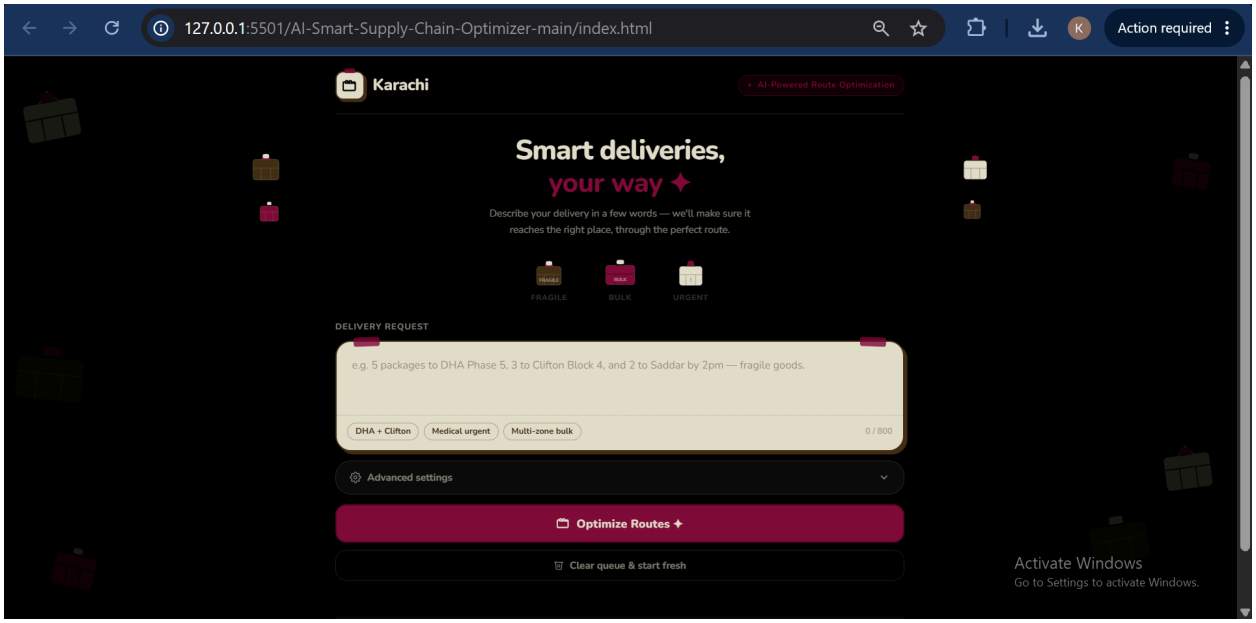
Category	Technology	Purpose
Language	Python 3.10+	Core runtime
Web Framework	FastAPI	REST API with async support
LLM Inference	Groq API (LLaMA-3.3-70b)	NLP parsing and summarisation
Mapping	OpenRouteService API	Real-world distance and routing
Database	MongoDB (pymongo)	Delivery input persistence
Data Validation	Pydantic v2	Request / response schema enforcement
Frontend	Vanilla HTML/CSS/JS	Single-page UI served by FastAPI
Dependencies	python-dotenv, openai-client	Configuration and Groq SDK

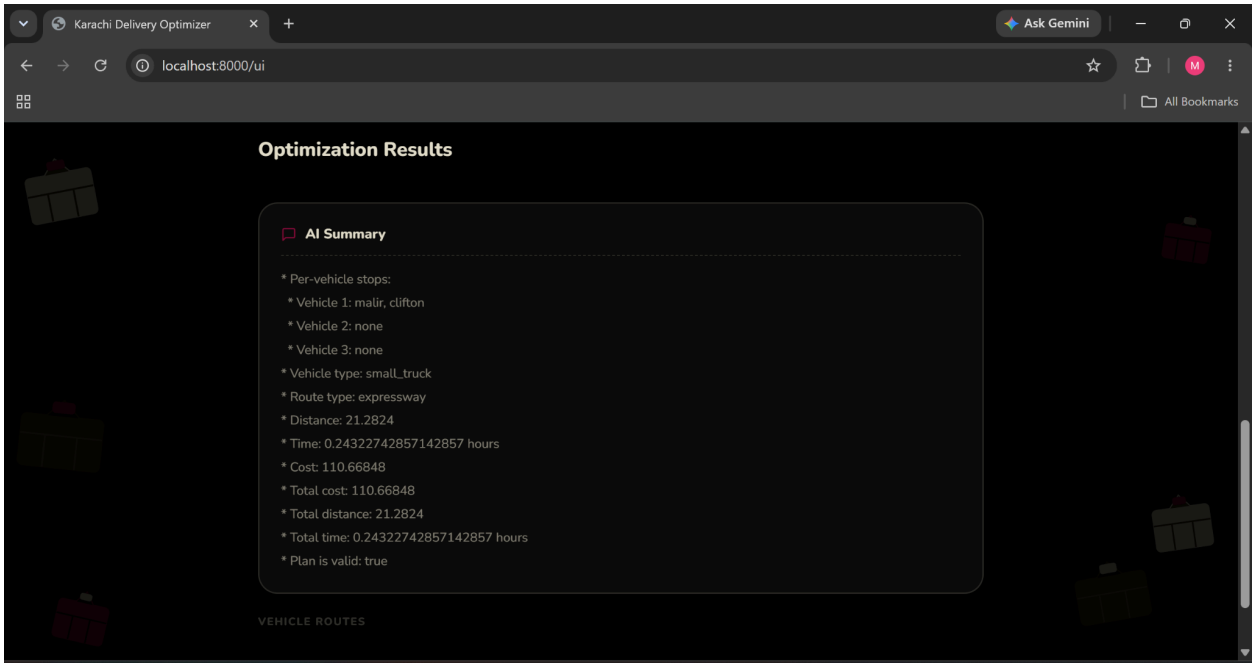
9. CONCLUSION

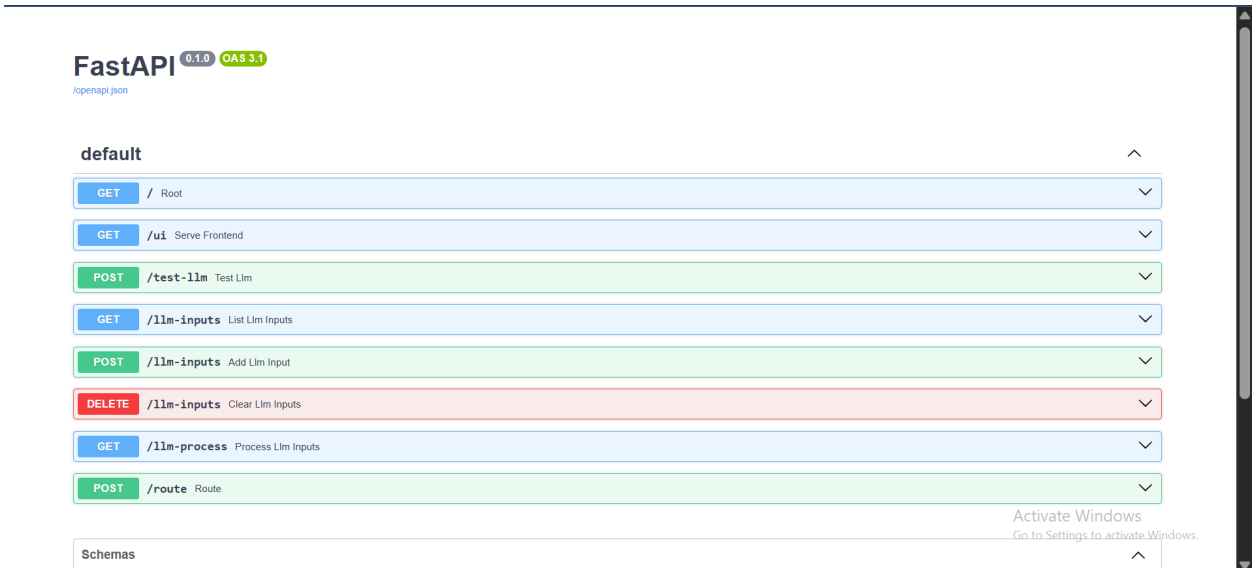
The AI Smart Supply Chain Optimizer is a technically ambitious and well-executed project. It successfully integrates four distinct AI paradigms — LLM-based natural language processing, heuristic graph search, evolutionary optimisation, and constraint satisfaction — into a coherent, production-oriented backend. The real-world API integrations, robust error handling, and multi-run GA strategy demonstrate engineering maturity beyond typical academic work.

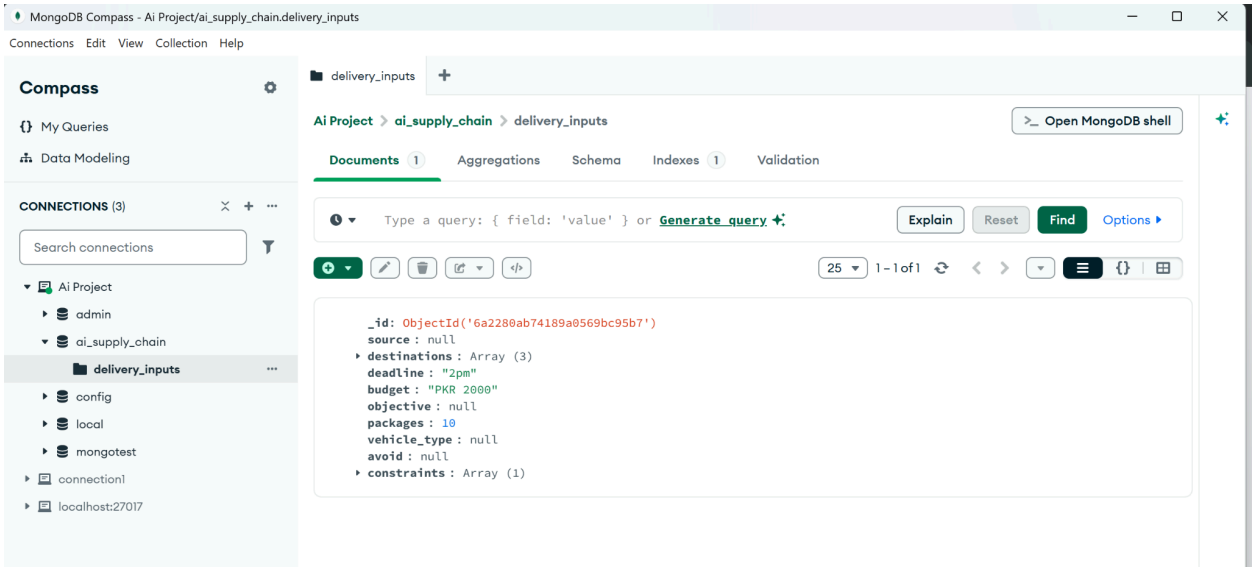
Primary recommendations are to unify shared constants into a single configuration module, add unit tests for the algorithmic core, and migrate ORS HTTP calls to async to avoid blocking the event loop under load.

Screenshots






FastAPI
0.1.0
OAS 3.1
/openapi.json
default
GET / Root
GET /ui Serve Frontend
POST /test-llm Test Llm
GET /llm-inputs List Llm Inputs
POST /llm-inputs Add Llm Input
DELETE /llm-inputs Clear Llm Inputs
GET /llm-process Process Llm Inputs
POST /route Route
Activate Windows
Go to Settings to activate Windows.
Schemas


MongoDB Compass - Ai Project/ai_supply_chain.delivery_inputs
Connections Edit View Collection Help
Compass
My Queries
Data Modeling
CONNECTIONS (3)
Search connections
Ai Project
admin
ai_supply_chain
delivery_inputs
config
local
mongotest
connection1
localhost:27017
delivery_inputs
Ai Project > ai_supply_chain > delivery_inputs
Open MongoDB shell
Documents 1
Aggregations
Schema
Indexes 1
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
25
1 - 1 of 1
_id: ObjectId('6a2280ab74189a0569bc95b7')
source : null
destinations : Array (3)
deadline : "2pm"
budget : "PKR 2000"
objective : null
packages : 10
vehicle_type : null
avoid : null
constraints : Array (1)